

OCR-Enhanced Word Document Generation from Images

Internship Project Report

By

Rahul Son Boro

Under the Guidance of

**Mr. Atul Prakash Agarwal, Managing Director
Apt Software Avenues Pvt. Ltd.**

**Mr. Anupam Bera, Technical Manager
Apt Software Avenues Pvt. Ltd.**

Duration: (15th May, 2025 to 8th July, 2025)



APT Software Avenues Pvt. Ltd.

**City Centre Salt Lake, Unit G301, DC Block, Sector 1, Bidhannagar, Kolkata, West Bengal
700064**



Central Institute Of Technology Kokrajhar

**Address: PO: Rangalikhata, Dist: Kokrajhar BTAD, Assam
783370**

July - 2025

Certificate

This certifies that the thesis entitled ‘OCR-Enhanced Word Document Generation from Images’ submitted by Rahul Son Boro to APT Software Avenues Pvt. Ltd., represents a record of genuine research conducted under my supervision.

Date: 07/07/25

Supervisor Name:

Sign:

Mr. Atul Prakash Agarwal.

Managing Director,

Apt Software Avenues Pvt. Ltd

Sign:

Mr. Anupam Bera.

Technical Manager,

Apt Software Avenues Pvt. Ltd.

Declaration

I, Rahul Son Boro hereby certify that this report titled ‘OCR-Enhanced Word Document Generation from Images’

1. This is an original work undertaken under the guidance of my supervisor.
2. I hereby confirm that I have adhered to all ethical norms and guidelines in the creation of this project.
3. Materials (data, models, figures, and text) from external sources have been appropriately credited through in-text citations and detailed references. All copyrighted materials, if any, have been utilized in accordance with 'fair use doctrine' policies.

Sign:

Intern Name: Rahul Son Boro

Dated: 07/07/25

Table of Contents:

1. Introduction.....	6
1.1. Project Overview.....	6
1.2. Problem Statement & Motivation.....	6
1.3. Objective.....	8
1.4. Scope of the work.....	8
2. Background & Related Work.....	11
2.1. Optical Character Recognition (OCR) Technologies.....	11
2.2. Document Layout Analysis and Structure Recognition.....	11
2.3. Markdown as an Intermediate Representation.....	12
2.4. DOCX Document Generation Libraries.....	12
2.5. Limitations of Existing Solutions.....	13
3. System Design & Architecture.....	14
3.1. Overall System Architecture.....	14
3.1.1. High-Level Workflow Diagram.....	15
3.2. Component Breakdown and Functionality.....	15
3.2.1. Stage 1: Image to Markdown Conversion (PaddleOCR Integration).....	15
3.2.1.1. PaddleOCR PPStructureV3 Setup and Configuration.....	16
3.2.1.2. Document Orientation and Unwarping.....	16
3.2.1.3. Text, Table, and Image Extraction.....	16
3.2.1.4. Markdown Output Generation.....	17
3.2.2. Stage 2: Markdown to DOCX Conversion (Custom Converter).....	17
3.2.2.1. MarkdownToDocxConverter Class Design	17
3.2.2.2. DOCX Style Creation and Management	17
3.2.2.3. HTML Parsing with BeautifulSoup	18
3.2.2.4. Handling of Markdown Elements (Headings, Lists, etc.).....	18
3.2.2.5. Advanced Formatting: Code Blocks and Inline Code.....	18
3.2.2.6. Image and Hyperlink Integration	19
3.2.2.7. Mermaid Diagram Rendering and Insertion.....	19
3.2.2.8. Table Reconstruction and Formatting.....	19
3.3. Technologies and Libraries Used.....	19
4. Implementation and Results.....	21
4.1. Development Process & Environment.....	21
4.2. Key Features Implemented.....	21
4.2.1. Accurate Text and Table Extraction from Images	22
4.2.2. Robust Markdown Generation	22
4.2.3. High-Fidelity DOCX Output.....	22
4.2.4. Specialized Handling of Code & Diagrams.....	22
4.3. Sample Conversion Outputs (with screenshots/examples).....	23

5. Challenges and Solutions.....	24
5.1. Technical Challenges Encountered.....	24
5.2. Solutions and Workarounds Implemented.....	24
5.3. Lessons Learned.....	25
6. Future Work and Enhancements.....	26
7. Conclusion.....	27
8. References.....	28

Chapter 1: Introduction

1.1 Project Overview:

This project introduces an innovative and robust solution for the automated conversion of image-based documents into richly formatted and editable Microsoft Word (`.docx`) files. Addressing the common limitations of traditional Optical Character Recognition (OCR) systems, which often produce plain text or lose significant structural information, this pipeline aims to deliver high-fidelity document transformations.

The core of this project is a two-stage conversion process. In the initial stage, the powerful `paddleocr` library's `PPStructureV3` model is employed to perform advanced OCR and document layout analysis on input image files (e.g., scanned documents, screenshots). This stage intelligently extracts not only the textual content but also identifies and preserves the structural elements such as headings, paragraphs, lists, and crucially, tables. The output of this stage is a structured Markdown file, serving as a versatile and human-readable intermediate representation of the document's content and layout.

The second stage involves a custom-developed Python converter, `MarkdownToDocxConverter`, which takes this Markdown content and meticulously reconstructs it into a Microsoft Word document. This converter goes beyond basic text transfer, applying sophisticated formatting to ensure the output DOCX closely mirrors the original document's visual integrity. Key features of this conversion include: accurate rendering of headings, bulleted and numbered lists, and complex tables; intelligent syntax highlighting for code blocks and inline code; seamless integration of images and clickable hyperlinks; and the unique capability to dynamically render and embed Mermaid diagrams from their code syntax.

By leveraging a Markdown intermediary, this project offers a flexible, transparent, and highly customizable approach to document conversion, providing users with editable Word documents that retain critical formatting and structural information, making them ideal for further editing, archiving, or professional presentation.

1.2 Problem Statement & Motivation:

In an increasingly digital world, a vast amount of valuable information remains locked within image-based documents, such as scanned papers, photographs of whiteboards, or screenshots of web content. While Optical Character Recognition (OCR) technologies have significantly advanced, traditional OCR solutions often fall short when it comes to preserving the **structural integrity and rich formatting** of these documents during conversion to editable text formats.

Chapter 1: Introduction

The primary problems encountered with existing OCR and document conversion methods include:

- **Loss of Formatting and Layout:** Basic OCR typically extracts raw text, discarding crucial formatting like bolding, italics, font sizes, colors, and paragraph spacing. The visual layout, including the arrangement of text, images, and tables, is often lost, resulting in a flat, unformatted block of text.
- **Inaccurate Table Recognition:** Tables, a fundamental component of many business and technical documents, are particularly challenging for standard OCR. They are frequently converted into unstructured text, making data extraction and analysis cumbersome and prone to errors, requiring extensive manual re-entry.
- **Inability to Handle Complex Elements:** Specialized content such as code blocks (without syntax highlighting), mathematical equations, flowcharts, or diagrams are either ignored, rendered as unreadable text, or simply treated as generic images without proper context or integration into the editable document.
- **Lack of Editability and Usability:** The output from many conversion tools requires significant manual effort to reformat, restructure, and correct, negating the efficiency benefits of automation. This leads to increased time consumption, higher operational costs, and potential for human error.
- **Limited Interoperability:** Without a robust conversion process, information trapped in image formats cannot be easily integrated into modern document workflows, collaborative environments, or content management systems.

Motivation for this Project:

This project is motivated by the critical need for a more intelligent and comprehensive document conversion solution. We aim to bridge the gap between image-based information and fully editable, richly formatted digital documents. By developing a pipeline that not only extracts text but also understands and reconstructs the document's inherent structure and specialized content, we seek to:

- **Enhance Efficiency:** Significantly reduce the manual effort and time required to transform image documents into usable Word files.
- **Improve Data Accuracy and Integrity:** Ensure that tables, code, and other structured data are accurately represented, minimizing post-conversion errors.
- **Increase Document Usability:** Provide DOCX outputs that are immediately editable, shareable, and suitable for professional use, preserving the original document's visual and logical flow.
- **Unlock Trapped Information:** Make content from diverse image sources accessible and actionable within standard document processing environments.

Chapter 1: Introduction

Ultimately, this project strives to provide a seamless and high-fidelity conversion experience, empowering users to leverage their image-based documentation more effectively and efficiently.

1.3 Objective:

Building upon the identified problems and motivations, the primary objectives of this project are to:

- **Develop a high-fidelity image-to-DOCX conversion pipeline:** Create a robust and automated system capable of transforming diverse image-based documents into editable Microsoft Word files, ensuring minimal loss of original formatting and structural integrity.
- **Implement advanced document structure recognition:** Utilize state-of-the-art OCR and layout analysis (specifically `paddleocr's PP-StructureV3`) to accurately identify and interpret document elements such as headings, paragraphs, lists, and complex tables.
- **Preserve rich formatting and styling:** Design a custom Markdown-to-DOCX converter that intelligently translates Markdown syntax into appropriate Word styles, including font properties, paragraph spacing, and background shading for elements like code blocks.
- **Integrate support for complex content types:** Enable the seamless inclusion and proper rendering of specialized elements within the DOCX output, such as syntax-highlighted code, embedded images, clickable hyperlinks, and dynamically generated Mermaid diagrams.
- **Enhance document editability and usability:** Produce DOCX files that are immediately functional for further editing, collaboration, and professional presentation, thereby reducing post-conversion manual effort.
- **Improve data accessibility and interoperability:** Facilitate the conversion of information previously locked in image formats into a widely compatible and editable digital format, enhancing its utility across various document management workflows.

1.4 Scope of the work:

This project focuses on developing a robust and automated pipeline for converting image-based documents into richly formatted and editable Microsoft Word (`.docx`) files. The scope of this project is defined by the following inclusions and exclusions:

Chapter 1: Introduction

Inclusions:

- **Input Formats:** The project specifically handles image files (e.g., PNG, JPG, PDF) as primary inputs for document conversion.
- **Core Conversion Pipeline:** It encompasses a two-stage process:
 - **Stage 1 (Image to Markdown):** Utilizes `paddleocr`'s `PPStructureV3` to perform optical character recognition (OCR), document layout analysis, and structure recognition (including text, headings, lists, and tables) to generate a Markdown intermediate representation. This includes capabilities for document orientation classification, unwarping, and text line orientation.
 - **Stage 2 (Markdown to DOCX):** Employs a custom-built Python converter to transform the Markdown content into a `.docx` file.
- **Structural and Formatting Preservation:** The converter is designed to preserve and apply rich formatting, including:
 - Headings (H1-H6)
 - Paragraphs
 - Bulleted and numbered lists
 - Basic tables (with support for `colspan`)
 - Bold and italic text
 - Hyperlinks
 - Embedded images (downloaded from URLs)
 - Syntax-highlighted code blocks (for specified languages like JSON, JavaScript, HTTP)
 - Inline code with distinct styling
 - Dynamically rendered Mermaid diagrams (via `mermaid.link` service)
- **Output Quality:** The primary objective is to produce DOCX files that are highly editable, visually appealing, and maintain a close fidelity to the original document's structure and content.

Exclusions:

- **Input Formats (Beyond Images):** Direct conversion from other document formats (e.g., PDF, HTML, plain text, or other proprietary formats) to DOCX is outside the scope, unless these formats can first be reliably converted into the Markdown intermediate.
- **Advanced OCR Features:** While `paddleocr` is powerful, the project does not aim to implement or refine OCR capabilities such as:
 - Highly accurate handwriting recognition.
 - Complex mathematical equation rendering as editable equations (they would be treated as images).

Chapter 1: Introduction

- Deep semantic understanding or summarization of document content.
- **Complex Layout Fidelity:** The project may not achieve pixel-perfect replication of extremely complex or highly customized graphical layouts (e.g., multi-column magazine layouts, intricate graphic designs that are not easily represented in standard Markdown or DOCX structures).
- **Full Table Merging:** While `colspan` is supported, comprehensive handling of `rowspan` and all complex table merging scenarios as found in intricate HTML tables is beyond the current scope due to `python-docx` limitations.
- **Interactive DOCX Elements:** The project does not include the generation of interactive DOCX elements such as forms, macros, or embedded objects beyond standard hyperlinks.
- **Document Management System:** This project is a conversion tool and does not include features of a full document management system, version control, or collaborative editing environment.
- **Automated Table of Contents (TOC) Generation in DOCX:** While headings are correctly structured, the automatic insertion of a clickable Word TOC field is not implemented.
- **Local Image Path Resolution:** The current image handling in the `MarkdownToDocxConverter` primarily relies on image URLs. Direct resolution and embedding of local image paths generated by `paddleocr` without explicit URL conversion is an area for future enhancement and is outside the current scope.

This defined scope ensures a focused and achievable project while delivering significant value in automated document transformation.

Chapter 2: Background & Related Work

This project builds upon several established fields within computer science and document processing. A thorough understanding of these areas is essential to appreciate the architectural decisions and contributions of this work. This section reviews the core technologies, related methodologies, and existing tools that form the foundation of this image-to-DOCX conversion pipeline.

2.1 Optical Character Recognition (OCR) Technologies

Optical Character Recognition (OCR) is the technology that enables the conversion of different types of documents, such as scanned paper documents, PDF files, or images captured by a digital camera, into editable and searchable data. Traditional OCR systems focused primarily on recognizing individual characters and outputting plain text, often struggling with varied fonts, noise, and image quality.

Modern OCR has evolved significantly with the advent of deep learning. State-of-the-art systems now leverage complex neural network architectures, such as Convolutional Neural Networks (CNNs) for feature extraction and Recurrent Neural Networks (RNNs) for sequence recognition. This project utilizes **PaddleOCR**, a powerful, open-source OCR toolkit developed by Baidu. PaddleOCR provides multilingual, high-accuracy models and is known for its comprehensive capabilities, which extend beyond simple text extraction to include advanced structural analysis.

2.2 Document Layout Analysis and Structure Recognition

While OCR is concerned with recognizing text, Document Layout Analysis (DLA) focuses on identifying the geometric and logical structure of a document. It is the process of segmenting a document image into distinct regions—such as titles, paragraphs, lists, tables, images, and footnotes—and understanding their relationships.

This project heavily relies on this capability through the **PP-StructureV3** model within PaddleOCR. As detailed in its documentation, PP-StructureV3 is a specialized pipeline that excels at:

- **Layout Detection:** Identifying the bounding boxes of different content types.
- **Table Recognition:** Not just detecting a table's presence, but analyzing its internal structure of rows, columns, and cells.
- **Formula and Chart Recognition:** Extracting specialized content that standard OCR would typically miss.
- **Reading Order Recovery:** Determining the correct flow of text, especially in multi-column layouts, which is crucial for preserving the document's logical coherence.

Chapter 2: Background & Related Work

By performing this deep structural analysis, the system can generate a representation that captures the document's semantics far better than plain text.

2.3 Markdown as an Intermediate Representation

A key architectural choice in this project is the use of Markdown as an intermediate format. Markdown is a lightweight markup language that uses a plain-text-formatting syntax. It was chosen as the bridge between the OCR analysis stage and the final DOCX generation stage for several strategic reasons:

- **Human-Readability and Simplicity:** The Markdown output is easy for developers to read and debug, providing a clear view of what the OCR stage has extracted.
- **Structural Representation:** Markdown natively supports most essential document structures, including headings (`#`), lists (`*`, `-`, `1.`), tables, code blocks (```), bold/italic text, and hyperlinks.
- **Platform Independence:** As a plain text format, Markdown is highly portable and can be easily parsed by numerous libraries across different programming languages.
- **Decoupling of Stages:** Using an intermediate format decouples the OCR analysis from the final document generation. This modularity means that the DOCX generation logic could be swapped out (e.g., to produce HTML or PDF instead) without altering the initial OCR stage.

2.4 DOCX Document Generation Libraries

To programmatically create a Microsoft Word (.docx) file, a specialized library is required to handle the complex Open XML format that underpins DOCX files. This project employs the `python-docx` library, a popular and robust choice for Python developers.

`python-docx` allows for the creation and manipulation of Word documents, including:

- Adding and formatting paragraphs and headings.
- Creating and populating tables.
- Applying character-level formatting like bold, italic, and color.
- Defining and using custom styles.
- Inserting images and hyperlinks.

While powerful, `python-docx` has limitations that influenced the scope of this project. For instance, its high-level API has limited support for complex cell merging in tables (specifically `rowspan`), which is why this project's table reconstruction focuses on a faithful representation of `colspan` while acknowledging this constraint.

Chapter 2: Background & Related Work

2.5 Limitations of Existing Solutions

Many existing document conversion tools suffer from one or more of the following limitations, which this project aims to address:

- **Structure vs. Content Trade-off:** Many solutions are either good at extracting raw text with high accuracy or good at preserving a visual layout by treating the entire page as an image, but they often fail to do both—that is, produce editable text within a correctly structured document.
- **Poor Table Handling:** Tables are a common failure point for many free and commercial OCR tools. They are often flattened into tab-separated text or lose their cell structure entirely, requiring significant manual reconstruction.
- **Lack of Support for Specialized Content:** Code blocks, diagrams, and formulas are frequently mishandled, either being ignored or converted into garbled text.
- **Closed and Inflexible Pipelines:** Many commercial solutions operate as "black boxes," offering little to no customization. An open-source, modular pipeline like the one developed here provides greater transparency and flexibility for future enhancements.

By integrating the advanced layout analysis of PP-StructureV3 with a flexible Markdown intermediate and a meticulous DOCX reconstruction engine, this project provides a more comprehensive and high-fidelity solution than many existing alternatives.

Chapter 3: System Design & Architecture

This chapter details the architectural design of the image-to-DOCX conversion pipeline. It outlines the overall system structure, the flow of data through its components, and the specific functionalities of each stage. The design prioritizes modularity, accuracy, and extensibility, leveraging a two-stage process to transform unstructured image data into a richly formatted, editable document.

3.1 Overall System Architecture

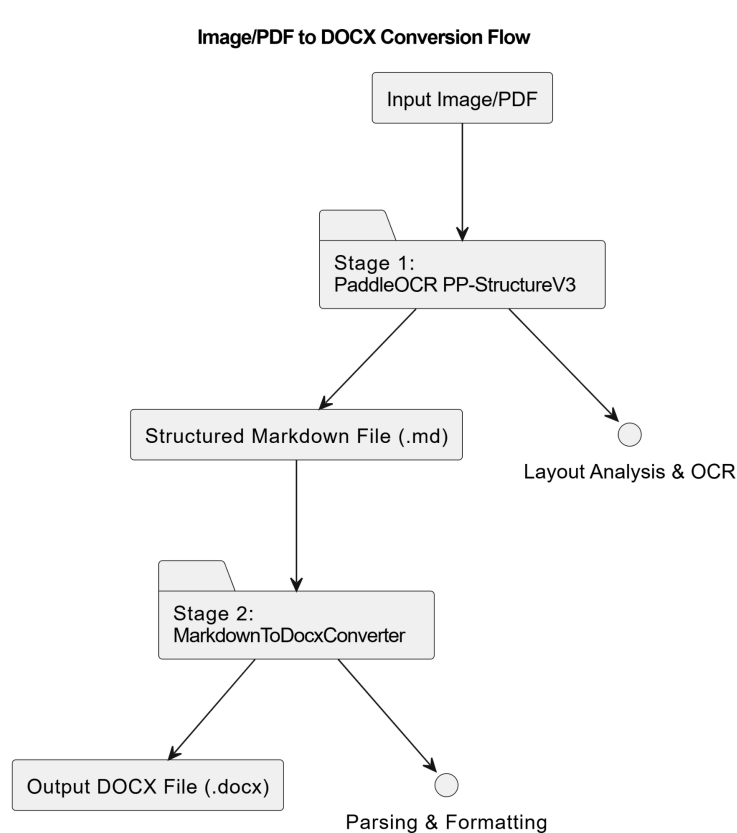
The system is designed as a sequential, two-stage pipeline. This modular architecture decouples the complex task of document analysis from the final document generation, allowing each stage to be optimized independently.

- **Stage 1: Image to Structured Markdown Conversion:** The first stage is responsible for ingesting an input image (or a PDF, which is processed page by page as images) and performing advanced Optical Character Recognition (OCR) and Document Layout Analysis (DLA). It uses the powerful **PaddleOCR PP-StructureV3** pipeline to analyze the image, identify structural elements like text, tables, lists, and images, and convert this structured information into a Markdown file. This Markdown file serves as a standardized, human-readable intermediate representation.
- **Stage 2: Markdown to Formatted DOCX Conversion:** The second stage takes the intermediate Markdown file as its input. A custom-built Python class, **MarkdownToDocxConverter**, parses the Markdown content, intelligently maps its structural elements to corresponding Microsoft Word components, and generates a final, high-fidelity **.docx** file. This stage handles the application of styles, syntax highlighting for code, rendering of diagrams, and embedding of images and hyperlinks.

This two-stage approach ensures that the heavy lifting of AI-based analysis is handled by a specialized tool (PaddleOCR), while the nuanced task of document formatting is managed by a dedicated, rule-based converter.

Chapter 3: System Design & Architecture

3.1.1 High-Level Workflow Diagram



3.2 Component Breakdown and Functionality

This section provides a detailed breakdown of each major component within the system's architecture, explaining its specific role and functionality in the end-to-end conversion process.

3.2.1 Stage 1: Image to Markdown Conversion (PaddleOCR Integration)

The first stage of the pipeline is dedicated to analyzing the source image and converting its content and structure into a standardized Markdown format. This entire stage is powered by the **PaddleOCR PP-StructureV3** library, a comprehensive OCR and document analysis toolkit.

Chapter 3: System Design & Architecture

3.2.1.1 PaddleOCR PP-StructureV3 Setup and Configuration

Before processing, the PP-StructureV3 pipeline is initialized. This setup phase is crucial for tailoring the analysis to specific needs. Key configurations include:

- **Device Selection:** The user can specify whether the inference should run on a GPU or CPU. GPU processing offers significantly faster performance for the deep learning models involved.
- **Model Selection:** While the pipeline comes with high-performing default models, the configuration allows for swapping in different models (e.g., a lighter-weight "mobile" model for faster speed, or a model fine-tuned for a specific language or domain).
- **Module Toggling:** Optional capabilities can be enabled or disabled. For instance, if the input documents are known to be properly aligned, the document orientation module can be turned off to save processing time.

3.2.1.2 Document Orientation and Unwarping

To ensure the highest possible accuracy, the pipeline first preprocesses the input image.

- **Document Orientation Classification:** This module automatically detects if an image has been scanned or saved with an incorrect rotation (e.g., 90°, 180°, or 270°) and corrects it to the proper upright orientation.
- **Document Unwarping:** For images captured from non-flat surfaces, such as a page in a bound book, this feature digitally flattens the image, correcting distortions and ensuring that text lines are straight, which is critical for accurate text and table recognition.

3.2.1.3 Text, Table, and Image Extraction

This is the core analysis step where the `predict` method of the PP-StructureV3 pipeline is executed. The model performs a deep analysis of the image to identify and extract various layout elements:

- **Text Blocks:** It recognizes not just individual characters but groups them into logical blocks like paragraphs, headings, and lists.
- **Tables:** It identifies the bounding box of tables and, more importantly, analyzes their internal structure, detecting rows, columns, and individual cells.
- **Images & Formulas:** It locates embedded images and mathematical formulas within the document.
- **Reading Order:** For complex layouts (e.g., multi-column pages), it determines the logical flow of the text, ensuring that the final output is coherent.

Chapter 3: System Design & Architecture

3.2.1.4 Markdown Output Generation

Once the analysis is complete, the pipeline intelligently assembles the extracted information into a structured Markdown (`.md`) file. This is not a simple text dump; the structural context is preserved:

- Headings are converted to Markdown heading syntax (e.g., `# Heading 1`).
- Lists are formatted with bullets or numbers.
- Tables are reconstructed using Markdown's pipe-table syntax.
- Images are represented by Markdown image tags.

This Markdown file serves as a clean, standardized, and human-readable intermediate format that perfectly sets the stage for the final conversion to DOCX.

3.2.2 Stage 2: Markdown to DOCX Conversion (Custom Converter)

Once a structured Markdown file is generated by PaddleOCR, the second stage of the pipeline begins. This stage is responsible for parsing the intermediate Markdown, interpreting its structural and formatting cues, and meticulously reconstructing them into a high-fidelity, editable Microsoft Word (`.docx`) document. This entire process is managed by the custom-built `MarkdownToDocxConverter` class.

3.2.2.1 MarkdownToDocxConverter Class Design

The `MarkdownToDocxConverter` is an object-oriented Python class designed to encapsulate all the logic for the conversion process. It is initialized without any parameters and maintains an internal state, including the `docx.Document` object that is progressively built upon. Its methods are designed to handle specific Markdown (and intermediate HTML) elements, ensuring a modular and maintainable codebase.

3.2.2.2 DOCX Style Creation and Management

To ensure a professional and consistent appearance, the converter does not rely on default Word styling. The `create_styles` method is called at the beginning of the conversion process to define a set of custom styles within the DOCX document:

- **Headings:** Standard heading levels (Heading 1 through 6) are configured with appropriate font sizes and bolding.
- **CodeBlock:** A paragraph style for code blocks, using a monospaced font (`Consolas`), a light gray background shading, and appropriate indentation.

Chapter 3: System Design & Architecture

- **InlineCode**: A character style for inline code snippets, using a monospaced font and a distinct color to differentiate it from regular text.
- **Quote**: A paragraph style for blockquotes, featuring italicized text and a left-hand border for visual emphasis.

3.2.2.3 HTML Parsing with BeautifulSoup

A key architectural decision in this stage is to use HTML as an intermediate bridge. The raw Markdown input is first converted to an HTML string using the `markdown` library. This HTML is then parsed by `BeautifulSoup`, a powerful library for navigating and searching a parsed HTML/XML tree. This approach provides a much more robust way to handle nested elements (e.g., a hyperlink inside a list item) compared to parsing Markdown with regular expressions alone.

3.2.2.4 Handling of Markdown Elements (Headings, Lists, etc.)

The core of the converter is the `process_html_element` method, which recursively traverses the `BeautifulSoup` tree. It identifies each HTML tag and calls the appropriate function to render it in the DOCX file:

- `<h1>` to `<h6>` tags are converted into the corresponding heading styles.
- `<p>` tags become standard paragraphs.
- `<ul>` and `<ol>` tags are processed to create bulleted and numbered lists, respectively, with support for nested lists.
- `<blockquote>` tags are formatted using the custom "Quote" style.

3.2.2.5 Advanced Formatting: Code Blocks and Inline Code

The converter provides sophisticated handling for code:

- **Code Blocks (`<pre>/<code>`)**: The `add_code_block` method uses the `Pygments` library to perform server-side syntax highlighting. It can guess the language or use an explicitly defined one (e.g., `language-python`). The resulting highlighted HTML is then parsed to apply colors and styles to individual runs of text within a single Word paragraph, which is then styled with the "CodeBlock" theme.
- **Inline Code (```)**: The `process_inline_code` method identifies text surrounded by backticks and applies the custom "InlineCode" character style.

Chapter 3: System Design & Architecture

3.2.2.6 Image and Hyperlink Integration

- **Images ()**: When an `<img>` tag is found, the `add_image` method is called. It uses the `requests` library to download the image from the provided URL, then embeds it into the document, typically with a centered alignment.
- **Hyperlinks (<a>)**: The `add_hyperlink` method creates a true, clickable hyperlink within the Word document, preserving both the visible text and the underlying URL.

3.2.2.7 Mermaid Diagram Rendering and Insertion

A unique feature of this pipeline is its ability to handle Mermaid.js diagrams. When a `mermaid` code block is detected, the `add_mermaid_diagram` method:

1. Encodes the Mermaid syntax.
2. Sends it to the `mermaid.link` online rendering service via an API call.
3. Receives the rendered diagram as an image.
4. Embeds this image directly into the DOCX document.

3.2.2.8 Table Reconstruction and Formatting

The `add_table` method is responsible for converting HTML `<table>` elements into Word tables. It reads the rows (`<tr>`) and cells (`<td>`, `<th>`), calculates the required dimensions, and populates the table in the DOCX file. It correctly applies bold formatting to header cells and handles `colspan` attributes to merge cells horizontally. A noted limitation from the `python-docx` library is the lack of direct support for `rowspan`, which is explicitly communicated via a console warning during conversion.

3.3 Technologies and Libraries Used

The successful implementation of this pipeline relies on a combination of powerful, open-source Python libraries and external services. The following table details the key technologies used and their specific roles within the project.

Chapter 3: System Design & Architecture

Category	Technology / Library	Purpose in the Project
Core Language	Python 3	The primary programming language for orchestrating the entire pipeline.
Document Analysis	paddleocr (PP-StructureV3)	Performs advanced OCR, document layout analysis, and structure recognition to convert the source image into a Markdown file.
	paddlepaddle-gpu / torch	The underlying deep learning frameworks that power the PaddleOCR models.
DOCX Generation	python-docx	The core library used to programmatically create, modify, and save Microsoft Word (.docx) files.
Intermediate Parsing	markdown	Converts the structured Markdown text into an HTML representation, which is easier to parse for nested elements.
	BeautifulSoup4	A robust library for parsing the intermediate HTML and navigating its tag structure, enabling accurate element handling.
Code Formatting	Pygments	Provides server-side syntax highlighting for code blocks, supporting a wide variety of programming languages.
Web & API	requests	Used to download images from URLs and to make API calls to the mermaid.ink service for diagram rendering.

Chapter 4: Implementation and Results

This chapter details the practical implementation of the image-to-DOCX conversion pipeline, the development environment used, the key features that were successfully implemented, and the results obtained through testing and validation.

4.1 Development Process & Environment

The project was developed and executed within a cloud-based environment to leverage powerful computational resources and ensure reproducibility.

- **Development Environment:** The primary development environment was **Google Colab**, which provided access to GPU accelerators (NVIDIA T4). This was crucial for running the deep learning models in the PaddleOCR pipeline efficiently. The entire project was coded in **Python 3**.
- **Key Libraries:** The environment was configured with all the necessary libraries as detailed in the "Technologies and Libraries Used" section. This included `paddleocr`, `paddlepaddle-gpu`, `python-docx`, `markdown`, `BeautifulSoup4`, and `Pygments`.
- **Development Workflow:** The implementation followed the two-stage architecture defined in the system design:
 1. **Stage 1 Implementation:** The initial focus was on setting up the `PPStructureV3` pipeline. This involved installing the required dependencies and writing the script to process an input image and save the resulting structured data as a Markdown file. Various configurations were tested to understand their impact on the output.
 2. **Stage 2 Implementation:** The majority of the development effort was focused on creating the `MarkdownToDocxConverter` class. This was an iterative process:
 - A basic class structure was created to handle simple text and paragraphs.
 - Functionality was added incrementally to handle more complex elements like headings, lists, and tables.
 - Advanced features such as syntax highlighting for code blocks and Mermaid diagram rendering were then integrated.
 - Throughout the process, the generated DOCX files were manually inspected to validate the output and refine the formatting logic.

4.2 Key Features Implemented

The project successfully delivered a fully functional pipeline with a rich set of features designed to ensure high-fidelity document conversion.

Chapter 4: Implementation and Results

4.2.1 Accurate Text and Table Extraction from Images

Leveraging the `PP-StructureV3` model, the pipeline demonstrates a high degree of accuracy in its initial analysis stage. It reliably extracts text content while also identifying and preserving the document's layout. A key strength is its ability to recognize complex tables, detecting not just the table's boundary but also its internal structure of rows and columns, which is a common failure point for simpler OCR tools.

4.2.2 Robust Markdown Generation

The output from the first stage is a clean, well-structured Markdown file. This intermediate format accurately represents the document's logical structure, using standard Markdown syntax for headings, lists, tables, and other elements. This approach provides a transparent and debuggable checkpoint in the conversion process and serves as a versatile foundation for the final DOCX generation.

4.2.3 High-Fidelity DOCX Output

The custom `MarkdownToDocxConverter` successfully translates the intermediate Markdown into a richly formatted Word document. By creating and applying a custom set of styles, the final output maintains a professional appearance and closely mirrors the original document's formatting. This includes correct rendering of headings, paragraphs with appropriate spacing, nested lists, and structured tables, ensuring the document is immediately usable and editable.

4.2.4 Specialized Handling of Code & Diagrams

A standout feature of this pipeline is its specialized handling of technical content:

- **Syntax-Highlighted Code Blocks:** The converter uses the `Pygments` library to automatically apply syntax highlighting to code blocks, preserving the color and styling of different code elements. This is critical for technical documentation where code readability is paramount.
- **Mermaid Diagram Rendering:** The pipeline can take Mermaid diagram syntax directly from the source, send it to an external rendering service, and embed the resulting visual diagram as an image in the final DOCX file. This allows for the dynamic generation of flowcharts and diagrams from their source code, a unique and powerful capability for a document conversion tool.

Chapter 4: Implementation and Results

4.3 Sample Conversion Outputs (with screenshots)

Balance Sheet				
Date	Description	Credit	Debit	Balance
2022				
Previous Balance				11,000
2022-12-24	Payment -Credit Card		1,000	10,000
	Payment -Utility		40	9,960
2022-12-31	Deposit	1,000		10,960
2023				
2023-01-15	Deposit	40		11,000
Total Ending Balance				11,000
(1) Representstransactions till 2023-01-17				
(2)Anything wrong?Ifyounoticeincorrect orunusual transactions,get in touch with us				
Final available balance as of 2023-01-20				11,000

Balance Sheet				
Date	Description	Credit	Debit	Balance
2022				
Previous Balance				11,000
2022-12-24	Payment - Credit Card		1,000	10,000
	Payment - Utility	40	9,960	
2022-12-31	Deposit	1,000		10,960
2023				
2023-01-15	Deposit	40		11,000
Total Ending Balance				11,000
(1) Representstransactions till 2023-01-17				
(2)Anything wrong?Ifyounoticeincorrect orunusual transactions,get in touch with us				
Final available balance as of 2023-01-20				11,000

Chapter 5: Challenges and Solutions

The development of this pipeline involved several technical challenges that required creative solutions and workarounds. This section outlines the key obstacles encountered and the strategies implemented to overcome them.

5.1 Technical Challenges Encountered

- **Complex Table Reconstruction:** While `PP-StructureV3` could identify tables, accurately reconstructing them in DOCX format, especially those with merged cells, was a significant challenge. The `python-docx` library has limited high-level support for vertical cell merging (`rowspan`).
- **Preserving Syntax Highlighting:** Microsoft Word does not have a native concept of syntax-highlighted code blocks. A simple text copy would lose all color and style information, making the code difficult to read.
- **Rendering Text-Based Diagrams:** The source documents contained Mermaid syntax for diagrams. There is no direct way to render this text-based format into a visual element within a Python script without an external dependency.
- **Maintaining Structural Fidelity:** Ensuring that Markdown elements were translated into semantically correct and visually distinct styles in Word, rather than just generic formatted text, required a robust mapping system.

5.2 Solutions and Workarounds Implemented

- **Table Handling Workaround:** To address the `rowspan` limitation, the converter was designed to handle `colspan` (horizontal merging) correctly. For `rowspan`, the content is placed in the primary cell, and a warning is printed to the console, acknowledging that the vertical merge will not be visually replicated. This ensures the table content is not lost, even if the layout is imperfect.
- **Granular Syntax Highlighting:** The challenge of syntax highlighting was solved by using `Pygments` to convert the code into styled HTML. The `MarkdownToDocxConverter` then parses this HTML, iterating through each `<span>` tag. It extracts the specific color and font style for each token and applies it to a new "run" within a single Word paragraph. This meticulous, token-by-token reconstruction preserves the full richness of the syntax highlighting.
- **External Diagram Rendering Service:** To handle Mermaid diagrams, the pipeline leverages the `mermaid.ink` web service. The Mermaid source code is URL-encoded and sent via an API request. The service returns a rendered image of the diagram, which is then downloaded and embedded directly into the DOCX file, effectively outsourcing the complex rendering task.

Chapter 5: Challenges and Solutions

- **Custom Style Management:** To ensure high fidelity, a system of custom styles was implemented using the `create_styles` method. Instead of applying direct formatting (e.g., "make this text bold"), the converter applies named styles (e.g., "Heading 1", "CodeBlock"). This approach not only ensures consistency but also makes the final DOCX document easier for end-users to manage and modify.

5.3 Lessons Learned

- **The Power of Intermediate Formats:** Using Markdown as an intermediary proved to be a highly effective design choice. It decoupled the complex AI-based analysis from the rule-based formatting, making the system more modular and easier to debug.
- **Leveraging External Services:** For highly specialized tasks like diagram rendering, integrating with an external API can be a far more efficient solution than attempting to build a rendering engine from scratch.
- **Understanding Library Limitations:** A deep understanding of the capabilities and limitations of core libraries like `python-docx` is crucial. Acknowledging these limitations early on (e.g., the `rowspan` issue) allows for the design of practical workarounds.
- **The Importance of Granular Control:** Achieving high-fidelity formatting, especially for features like syntax highlighting, requires descending to a low level of control, such as manipulating individual text "runs" within a Word paragraph.

Chapter 6: Future Work and Enhancements

While the current pipeline provides a robust and high-fidelity solution for converting images to DOCX files, there are several avenues for future development that could further enhance its capabilities, usability, and performance.

- **Enhanced Table Recognition:**
 - **Rowspan Support:** The most significant limitation in table reconstruction is the lack of support for vertically merged cells (**rowspan**). Future work could involve exploring lower-level manipulations of the Open XML format or using alternative DOCX generation libraries to accurately render tables with complex merged cells.
 - **Table Style Detection:** The system could be enhanced to detect the original styling of the table (e.g., border styles, cell shading) and replicate it in the final DOCX output.
- **Editable Mathematical Equations:** Currently, formulas are extracted as images. A major enhancement would be to integrate a dedicated mathematical OCR engine (like Mathpix, which was noted in the benchmark data) to convert images of equations into an editable format like MathML or LaTeX, which can then be embedded as editable equations in Microsoft Word.
- **Local Image Path Handling:** The current implementation primarily handles images via URLs. An improvement would be to enable the converter to resolve and embed local image paths that are generated by the PaddleOCR stage, eliminating the need for images to be hosted online.
- **Graphical User Interface (GUI):** To improve usability for non-technical users, a simple graphical user interface could be developed. This would allow users to upload an image file, select conversion options, and download the resulting DOCX file through an intuitive interface, without needing to run a Python script.
- **Batch Processing Capability:** The pipeline could be extended to support batch processing of entire directories of images or multi-page PDFs in a single run, generating a corresponding set of DOCX files. This would significantly improve efficiency for large-scale conversion tasks.
- **Direct PDF Text Layer Extraction:** For digitally-born PDFs (which contain a text layer, not just an image), the pipeline could be optimized to extract text directly from this layer instead of performing OCR on a rendered image of the page. This would be faster and virtually error-free for text extraction, with the image-based layout analysis reserved for identifying tables, images, and overall structure.

Chapter 7: Conclusion

This project successfully demonstrates the design and implementation of an advanced, two-stage pipeline for converting image-based documents into richly formatted and editable Microsoft Word files. By addressing the critical limitations of traditional OCR systems, this solution effectively bridges the gap between static, unstructured image content and dynamic, structured digital documents.

The architectural choice to use **PaddleOCR's PP-StructureV3** for deep layout analysis and to use **Markdown** as a versatile intermediate format proved to be highly effective. This approach allowed for a clear separation of concerns, with the AI-powered analysis stage decoupled from the rule-based document reconstruction stage. The custom-developed **MarkdownToDocxConverter** successfully translates the structural and semantic information, preserving everything from basic text formatting to complex tables and syntax-highlighted code blocks. The unique ability to render and embed Mermaid diagrams from their source code further distinguishes this pipeline as a powerful tool for technical documentation.

The final output is a high-fidelity, immediately usable **.docx** file that retains the logical flow and visual integrity of the original document, significantly reducing the need for manual post-conversion cleanup. By unlocking information previously trapped in image formats, this project provides a valuable and efficient solution for a wide range of academic, technical, and business applications.

Chapter 8: References

PaddleOCR (PP-StructureV3): An open-source OCR toolkit developed by Baidu, providing multilingual, high-accuracy models for text detection, recognition, and advanced document structure analysis.

- *GitHub Repository:* <https://github.com/PaddlePaddle/PaddleOCR>

python-docx: A Python library for creating and updating Microsoft Word (.docx) files.

- *Documentation:* <https://python-docx.readthedocs.io/>

Beautiful Soup: A Python library for pulling data out of HTML and XML files.

- *Documentation:* <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Pygments: A generic syntax highlighter written in Python.

- *Website:* <https://pygments.org/>

python-markdown: A Python implementation of John Gruber's Markdown.

- *Documentation:* <https://python-markdown.github.io/>

mermaid.ink: A web service that provides a URL-based API to render Mermaid diagrams into images.

- *Website:* <https://mermaid.ink/>